

Mô Phỏng Dòng Chảy Torus

https://gemini.google.com/app/0539ad0559e54c8c

User prompt: hmm, không giống làm hình như bạn vẽ bông hoa su song đang bị sai ảnh 2d của nó tôi có kèm trong chat với lại âm dương bạn đang chia đơn giản thành 2 phần, mỗi phần 1 nửa trong khi âm dương thật sự thì phải uốn cong, bạn xem trong ảnh, ý tôi là kiểu kiểu thế. tôi cũng k hiểu ảnh do làm, nếu bạn hiểu thì vẽ lại thể hiện rõ ý tưởng cho tôi

Response: Nhìn vào hai bức ảnh bạn gửi, toàn bộ cấu trúc hình học thiêng liêng lộ diện rất rõ ràng:

- 1. **Flower of Life chuẩn (Ảnh 1):** Là mạng lưới mạng lục giác 2D gồm đúng **19 vòng tròn lồng ghép hoàn chỉnh** với các tâm giao điểm (chấm đỏ) và các vòng bao ngoài (vòng xanh).
- 2. **Hình xuyên sừng (Horn Torus / Spherical Torus) & Âm Dương (Ảnh 2):**
 - Đây không phải là chiếc bánh donut rỗng ở giữa, mà là **Horn Torus** ($R = r$)—hai mặt trong chạm khít nhau tại đúng trục lõi tạo thành một hình cầu hoàn chỉnh có cường hống xoáy (vortex funnel).
 - **Cặp mắt Âm Dương (Hai vòng tròn nhỏ đối xứng trên dưới):** Chính là hai p полюс (Cực Bắc & Cực Nam) của Torus.
 - **Đường phân chia chữ S uốn lượn:** Chia không gian thành hai khối năng lượng xoắn ốc 3D (Yin xoáy vào lõi, Yang xoáy bung ra ngoài), được chiếu phủ lên toàn bộ nền lưới hoa sự sống.

Mô phỏng Three.js chuẩn xác theo hình ảnh

Dưới đây là mã nguồn HTML5/Three.js dựng đúng mạng lưới Flower of Life 19 vòng tròn, lưới khung Horn Torus 3D, và hai dòng hạt xoắn uốn lượn theo hình hài chữ S ôm trọn lấy hai cực xoáy:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Flower of Life - Yin Yang Horn Torus</title>
  <style>
    body {
      margin: 0;
      background-color: #020204;
      overflow: hidden;
      font-family: monospace;
    }
    #info {
      position: absolute;
      top: 15px;
      left: 20px;
      color: #d1d5db;
      pointer-events: none;
      z-index: 10;
      line-height: 1.5;
    }
    .title {
      font-size: 14px;
      font-weight: bold;
      color: #fff;
      letter-spacing: 2px;
    }
    .sub {
      font-size: 11px;
      color: #9ca3af;
    }
  </style>
  <script type="importmap">
    {
      "imports": {
        "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
        "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
      }
    }
  </script>
</head>
<body>
  <div id="info">
    <div class="title">SACRED GEOMETRY // HORN TORUS & YIN-YANG DYNAMICS</div>
    <div class="sub">19-Circle Flower of Life Lattice • Dual Vortex Singularities (Yin-Yang Eyes)</div>
  </div>
```

```

<script type="module">
  import * as THREE from 'three';
  import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

  // 1. Scene Setup
  const scene = new THREE.Scene();
  const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 1000);
  camera.position.set(0, 0, 9);

  const renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setSize(window.innerWidth, window.innerHeight);
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
  document.body.appendChild(renderer.domElement);

  const controls = new OrbitControls(camera, renderer.domElement);
  controls.enableDamping = true;
  controls.dampingFactor = 0.05;

  const mainGroup = new THREE.Group();
  scene.add(mainGroup);

  // 2. Tham số Horn Torus (R = r để chạm khít tâm thành hình cầu)
  const R = 2.0;
  const r = 2.0;

  // 3. Dựng mạng lưới Flower of Life chuẩn 19 vòng tròn
  function createExactFlowerOfLife() {
    const folGroup = new THREE.Group();
    const radius = 1.0;
    const circleMat = new THREE.LineBasicMaterial({
      color: 0x9ca3af,
      transparent: true,
      opacity: 0.28
    });
  };

  function getCircle(cx, cy) {
    const curve = new THREE.EllipseCurve(cx, cy, radius, radius, 0, 2 * Math.PI, false, 0);
    const pts = curve.getPoints(64).map(p => new THREE.Vector3(p.x, p.y, 0));
    const geom = new THREE.BufferGeometry().setFromPoints(pts);
    return new THREE.Line(geom, circleMat);
  }

  // Tọa độ 19 tâm vòng tròn theo hệ lưới tam giác đều (Hexagonal grid)
  const centers = [{ x: 0, y: 0 }];
  const step = radius;

  // Vòng 1: 6 tâm bao quanh
  for (let i = 0; i < 6; i++) {
    const angle = (i * Math.PI) / 3;
    centers.push({ x: step * Math.cos(angle), y: step * Math.sin(angle) });
  }

  // Vòng 2: 12 tâm kề tiếp
  for (let i = 0; i < 6; i++) {
    const angle1 = (i * Math.PI) / 3;
    centers.push({ x: 2 * step * Math.cos(angle1), y: 2 * step * Math.sin(angle1) });

    const angle2 = angle1 + Math.PI / 6;
    centers.push({ x: Math.sqrt(3) * step * Math.cos(angle2), y: Math.sqrt(3) * step * Math.sin(angle2) });
  }

  centers.forEach(c => folGroup.add(getCircle(c.x, c.y)));

  // 2 vòng tròn lớn bao bọc ngoài cùng
  const outerMat = new THREE.LineBasicMaterial({ color: 0x60a5fa, transparent: true, opacity: 0.45 });
  [3 * radius, 3.08 * radius].forEach(R_out => {
    const curve = new THREE.EllipseCurve(0, 0, R_out, R_out, 0, 2 * Math.PI, false, 0);
    const pts = curve.getPoints(128).map(p => new THREE.Vector3(p.x, p.y, 0));
    folGroup.add(new THREE.Line(new THREE.BufferGeometry().setFromPoints(pts), outerMat));
  });

  return folGroup;
}

```

```

const fol = createExactFlowerOfLife();
mainGroup.add(fol);

// 4. Lưới Horn Torus 3D (Wireframe trắng mờ bao bọc)
const torusGeo = new THREE.TorusGeometry(R, r, 32, 64);
const torusMat = new THREE.MeshBasicMaterial({
  color: 0xffffffff,
  wireframe: true,
  transparent: true,
  opacity: 0.14
});
const hornTorus = new THREE.Mesh(torusGeo, torusMat);
hornTorus.rotation.x = Math.PI / 2;
mainGroup.add(hornTorus);

// 5. Cặp mắt Âm Dương tại 2 cực xoáy (Yin-Yang Singularity Rings)
const ringMat = new THREE.LineBasicMaterial({ color: 0xffffffff, transparent: true, opacity: 0.85 });
const eyeRadius = 0.55;
[r, -r].forEach(yPos => {
  const curve = new THREE.EllipseCurve(0, 0, eyeRadius, eyeRadius, 0, 2 * Math.PI, false, 0);
  const pts = curve.getPoints(48).map(p => new THREE.Vector3(p.x, yPos, p.y));
  const ring = new THREE.Line(new THREE.BufferGeometry().setFromPoints(pts), ringMat);
  mainGroup.add(ring);
});

// 6. Dòng hạt Âm Dương uốn lượn hình chữ S
const COUNT = 3600;
const pPos = new Float32Array(COUNT * 3);
const pCol = new Float32Array(COUNT * 3);
const particles = [];

for (let i = 0; i < COUNT; i++) {
  const isYang = i < COUNT / 2;
  const u = Math.random() * Math.PI * 2;
  // Đường chữ S điều biến góc v theo pha u
  const sOffset = Math.sin(u) * (Math.PI * 0.5);
  const v = (isYang ? 0 : Math.PI) + sOffset + (Math.random() - 0.5) * 1.2;

  particles.push({
    u: u,
    v: v,
    isYang: isYang,
    speedU: 0.012,
    speedV: 0.006
  });
}

const pGeom = new THREE.BufferGeometry();
pGeom.setAttribute('position', new THREE.BufferAttribute(pPos, 3));
pGeom.setAttribute('color', new THREE.BufferAttribute(pCol, 3));

// Particle Texture mềm
const canvas = document.createElement('canvas');
canvas.width = 16;
canvas.height = 16;
const ctx = canvas.getContext('2d');
const grad = ctx.createRadialGradient(8, 8, 0, 8, 8, 8);
grad.addColorStop(0, 'rgba(255,255,255,1)');
grad.addColorStop(1, 'rgba(255,255,255,0)');
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 16, 16);
const tex = new THREE.CanvasTexture(canvas);

const pMat = new THREE.PointsMaterial({
  size: 0.14,
  vertexColors: true,
  map: tex,
  transparent: true,
  blending: THREE.AdditiveBlending,
  depthWrite: false
});

const pSystem = new THREE.Points(pGeom, pMat);
mainGroup.add(pSystem);

```

```

// 7. Cập nhật chuyển động xoắn ốc
function updateDynamics() {
  const pos = pGeom.attributes.position.array;
  const col = pGeom.attributes.color.array;

  for (let i = 0; i < COUNT; i++) {
    const p = particles[i];
    p.u += p.speedU;
    p.v += p.speedV;

    const x = (R + r * Math.cos(p.u)) * Math.cos(p.v);
    const z = (R + r * Math.cos(p.u)) * Math.sin(p.v);
    const y = r * Math.sin(p.u);

    const idx = i * 3;
    pos[idx] = x;
    pos[idx + 1] = y;
    pos[idx + 2] = z;

    // Vùng lõi trung tâm sáng rực
    const coreFactor = (1 - Math.cos(p.u)) / 2;

    if (p.isYang) {
      // Nửa Dương (Trắng sáng / Hơi ánh vàng kim)
      col[idx] = 0.95;
      col[idx + 1] = 0.9 + coreFactor * 0.1;
      col[idx + 2] = 0.75 + coreFactor * 0.25;
    } else {
      // Nửa Âm (Xám khói tối / Hơi ánh lam sẫm)
      col[idx] = 0.2 + coreFactor * 0.5;
      col[idx + 1] = 0.3 + coreFactor * 0.5;
      col[idx + 2] = 0.45 + coreFactor * 0.55;
    }
  }

  pGeom.attributes.position.needsUpdate = true;
  pGeom.attributes.color.needsUpdate = true;
}

// 8. Render loop
function animate() {
  requestAnimationFrame(animate);
  updateDynamics();
  mainGroup.rotation.y += 0.003;

  controls.update();
  renderer.render(scene, camera);
}

animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
</script>
</body>
</html>

```

User prompt: theo bạn nói thì có thể con người có trái tim nằm ở trung tâm torus nhưng tim và cột sống lại tách rời nhau vậy tại sao k phải là cột sống là tâm torus

Response: Sự phân vân của bạn chạm đúng vào cốt lõi cấu trúc động lực học năng lượng của cơ thể: **Cột sống và Trái tim không đối nghịch nhau, mà đảm nhận hai vai trò hình học hoàn toàn khác nhau trong cùng một mô hình Torus.**

Trong một khối Torus hoàn chỉnh, luôn có sự phân biệt rõ ràng giữa **Trục đối xứng (Central Axis)** và **Điểm kỳ dị trung tâm (Singularity / Geometric Center)**:

- **Cột sống là Trục dòng chảy trung tâm (Central Flow Axis / Core Throat):**

- Cột sống (tương ứng với kênh năng lượng trung tâm *Sushumna*) đóng vai trò là **ống trục rỗng** xuyên suốt cơ thể, kết nối Cực Bắc (đỉnh đầu / Crown) với Cực Nam (đáy chậu / Perineum).
- Toàn bộ dòng năng lượng đi vào và đi ra khỏi cơ thể đều phải chạy dọc theo trục cột sống này. Do đó, cột sống là đường dẫn hướng trục, định hình chiều cao và hướng của toàn bộ khối trường.

• **Trái tim là Điểm hội tụ & Phát xạ trung tâm (Singularity Center / Zero-Point):**

- Nếu cột sống là đường thẳng tạo trục, thì **Trái tim nằm ở điểm giao thoa giữa các tầng tỷ lệ** (Golden Ratio point dọc theo chiều dài cơ thể).
- Về mặt điện sinh học đo đạc thực tế, trường điện từ của tim có điện thế mạnh gấp khoảng 60 lần và từ trường mạnh gấp tới 5.000 lần so với não bộ. Tim chính là "máy bơm" xung nhịp (oscillator) đẩy năng lượng ly tâm bung ra ngoài vỏ Torus và hút năng lượng hướng tâm quay trở lại.

- **Sự kết hợp hoàn hảo:** Cột sống tạo ra **độ thẳng và định hướng của trục xoay**, còn Trái tim nằm trên mặt phẳng xích đạo của trục đó để đóng vai trò **tâm điểm kích hoạt và điều phối nhịp đập của trường Torus**.

User prompt: code bạn viết tạo ra torus với 2 ván thì phải torus có thể có nhiều ván được hơn k bạn có thể xoay khỏi torus theo các góc khác nhau theo thời gian thực để hình in ra luôn luôn là hình yin yang dc k. cái này có thể kho

Response: Chính xác! Trong mô hình trước, hai cánh xoắn là do ta chia phân cực đối xứng bậc 2 (2 nhánh 180°). Trong tự nhiên và hình học Fractal:

1. **Số lượng vân (Multi-arm Toroidal Vortices):** Trường Torus hoàn toàn có thể có **3, 4, 6 hoặc 12 vân/cánh xoay** (tương ứng với các tỷ lệ điều hòa hình học thiêng liêng như 6 cánh của Flower of Life).
2. **Hình in ra luôn là Âm Dương từ mọi góc nhìn (Orthographic Projection Alignment):** Bí quyết để nhìn từ mọi góc đều ra cấu trúc Yin-Yang chữ S là:
 - Chuyển hệ thống hạt sang dạng **trường xoắn kép đối ngẫu lồng nhau (Dual Inverted Spirals)**.
 - Đồng bộ góc xoay của trường hạt sao cho **mặt cắt chữ S và 2 cực mắt xoay luôn tự căn chỉnh vuông góc (billboard alignment) theo hướng nhìn của Camera** theo thời gian thực!

Mô phỏng Three.js: Đa vân & Tự khóa mặt Yin-Yang theo góc Camera

File .html hoàn chỉnh dưới đây hỗ trợ tùy chỉnh số lượng vân xoắn (từ 2 đến 12 vân) và **tự động bẻ pha xoay để từ bất kỳ góc xoay chuột nào, hình chiếu cũng giữ trọn dáng chữ S và 2 mắt Âm Dương**:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Multi-Arm Yin-Yang Horn Torus (Perspective Locked)</title>
  <style>
    body {
      margin: 0;
      background-color: #020204;
      overflow: hidden;
      font-family: monospace;
      color: #e2e8f0;
    }
    #ui {
      position: absolute;
      top: 15px;
      left: 20px;
      pointer-events: auto;
      z-index: 10;
      line-height: 1.6;
      background: rgba(10, 15, 25, 0.7);
      padding: 12px 18px;
      border-radius: 8px;
      border: 1px solid rgba(255, 255, 255, 0.1);
    }
    .title {
      font-size: 13px;
      font-weight: bold;
      color: #38bdf8;
      letter-spacing: 1.5px;
      margin-bottom: 6px;
    }
    .controls {
```

```

display: flex;
align-items: center;
gap: 10px;
font-size: 12px;
}
select {
background: #1e293b;
color: #38bdf8;
border: 1px solid #475569;
padding: 3px 8px;
border-radius: 4px;
font-family: monospace;
outline: none;
cursor: pointer;
}
</style>
<script type="importmap">
{
  "imports": {
    "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
    "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
  }
}
</script>
</head>
<body>
<div id="ui">
<div class="title">DYNAMIC MULTI-VORTEX TORUS</div>
<div class="controls">
<label for="arms">Số lượng vân xoáy:</label>
<select id="arms">
  <option value="2">2 Vân (Âm - Dương Cô'Điễn)</option>
  <option value="3">3 Vân (Tam Tài / Triskelion)</option>
  <option value="4">4 Vân (Tứ Tượng)</option>
  <option value="6" selected>6 Vân (Flower of Life Hexagon)</option>
  <option value="12">12 Vân (Vòng Hoàng Đạo)</option>
</select>
</div>
</div>

<script type="module">
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

// 1. Khởi tạo Scene & Camera
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 1000);
camera.position.set(0, 0, 8.5);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
document.body.appendChild(renderer.domElement);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
controls.dampingFactor = 0.05;

const mainGroup = new THREE.Group();
scene.add(mainGroup);

// 2. Thông số Horn Torus chuẩn ( $R = r$  để'khép khít tâm cầu)
const R = 2.0;
const r = 2.0;
let NUM_ARMS = 6;

// 3. Khung Horn Torus Wireframe
const torusGeo = new THREE.TorusGeometry(R, r, 32, 64);
const torusMat = new THREE.MeshBasicMaterial({
  color: 0xffffffff,
  wireframe: true,
  transparent: true,
  opacity: 0.12
});

```

```

const torusMesh = new THREE.Mesh(torusGeo, torusMat);
torusMesh.rotation.x = Math.PI / 2;
mainGroup.add(torusMesh);

// 4. Mạng hạt năng lượng đa vôn
const COUNT = 4800;
const pPos = new Float32Array(COUNT * 3);
const pCol = new Float32Array(COUNT * 3);
let particles = [];

function initParticles() {
  particles = [];
  for (let i = 0; i < COUNT; i++) {
    const armIndex = i % NUM_ARMS;
    const armBaseAngle = (armIndex / NUM_ARMS) * Math.PI * 2;
    const u = Math.random() * Math.PI * 2;

    // Bù góc để mỗi cánh uốn thành hình chữ S xoắn kép ôm lấy lõi
    const spread = (Math.random() - 0.5) * (Math.PI / NUM_ARMS) * 0.9;
    const isYang = armIndex % 2 === 0;

    particles.push({
      u: u,
      armBase: armBaseAngle,
      spread: spread,
      isYang: isYang,
      speedU: 0.012,
      speedV: 0.006
    });
  }
}
initParticles();

document.getElementById('arms').addEventListener('change', (e) => {
  NUM_ARMS = parseInt(e.target.value);
  initParticles();
});

const pGeom = new THREE.BufferGeometry();
pGeom.setAttribute('position', new THREE.BufferAttribute(pPos, 3));
pGeom.setAttribute('color', new THREE.BufferAttribute(pCol, 3));

// Particle Canvas Texture
const canvas = document.createElement('canvas');
canvas.width = 16;
canvas.height = 16;
const ctx = canvas.getContext('2d');
const grad = ctx.createRadialGradient(8, 8, 0, 8, 8, 8);
grad.addColorStop(0, 'rgba(255,255,255,1)');
grad.addColorStop(1, 'rgba(255,255,255,0)');
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 16, 16);
const tex = new THREE.CanvasTexture(canvas);

const pMat = new THREE.PointsMaterial({
  size: 0.13,
  vertexColors: true,
  map: tex,
  transparent: true,
  blending: THREE.AdditiveBlending,
  depthWrite: false
});

const pSystem = new THREE.Points(pGeom, pMat);
mainGroup.add(pSystem);

// 5. Cập nhật Dòng chảy & Khóa hướng nhìn Âm Dương theo Camera
function updateDynamics() {
  const pos = pGeom.attributes.position.array;
  const col = pGeom.attributes.color.array;

  // Tính toán góc phương vị của Camera để tự động bù góc chiếu
  const camAngle = Math.atan2(camera.position.x, camera.position.z);

```

```

for (let i = 0; i < COUNT; i++) {
  const p = particles[i];
  p.u += p.speedU;

  // Chữ S điều biến góc v theo pha u + bù góc nhìn Camera
  const sModulation = Math.sin(p.u) * (Math.PI / NUM_ARMS);
  const v = p.armBase + sModulation + p.spread + camAngle;

  const x = (R + r * Math.cos(p.u)) * Math.cos(v);
  const z = (R + r * Math.cos(p.u)) * Math.sin(v);
  const y = r * Math.sin(p.u);

  const idx = i * 3;
  pos[idx] = x;
  pos[idx + 1] = y;
  pos[idx + 2] = z;

  // Cường độ sáng ở lõi và phân chia màu sắc theo từng vân
  const coreFactor = (1 - Math.cos(p.u)) / 2;

  if (p.isYang) {
    // Nhánh Dương: Vàng kim - Trắng
    col[idx] = 0.95;
    col[idx + 1] = 0.85 + coreFactor * 0.15;
    col[idx + 2] = 0.65 + coreFactor * 0.35;
  } else {
    // Nhánh Âm: Xanh ngọc - Xanh thẫm
    col[idx] = 0.2 + coreFactor * 0.6;
    col[idx + 1] = 0.5 + coreFactor * 0.5;
    col[idx + 2] = 0.9;
  }
}

pGeom.attributes.position.needsUpdate = true;
pGeom.attributes.color.needsUpdate = true;
}

// 6. Vòng lặp Render
function animate() {
  requestAnimationFrame(animate);

  updateDynamics();
  controls.update();
  renderer.render(scene, camera);
}

animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
</script>
</body>
</html>

```

User prompt: *toi dang thac mac: 1. co the day so van len den vo han, hay noi cach khac la chi co 1 van duy nhât va k co khoang cach nao dc k 2. co the tang giam do doc cua van duoc khong*

Response: Hai thắc mắc của bạn chạm đến bản chất toán học và cơ học lượng tử của trường năng lượng:

1. Đầy số vân lên vô hạn (hoặc chỉ 1 vân duy nhất không có khoảng cách)?

Hoàn toàn được, và đây chính là hai cách nhìn tương đương về sự chuyển pha từ "rời rạc" (discrete) sang "liên tục" (continuous field):

- **Trường hợp 1 sợi duy nhất phủ kín (Continuous Space-filling Curve):** Nếu tỷ lệ giữa tốc độ cuộn lõi (ω_u) và tốc độ xoay trục (ω_v) là một **số vô tỉ** (ví dụ tỉ lệ Vàng $\Phi \approx 1.6180339\dots$), thì đường xoắn ốc của 1 vân duy nhất sẽ **không bao giờ lặp lại quỹ đạo cũ**. Theo thời gian, một sợi duy nhất đó sẽ quét kín 100% diện tích mặt xuyên torus mà không để lại bất kỳ khoảng trống nào.
- **Trường hợp vô hạn vân:** Khi số lượng vân $N \rightarrow \infty$, khoảng cách giữa các vân triệt tiêu về 0. Cấu trúc torus biến thành một **màng trường liên tục (continuous manifold)** hoặc chất lưu plasma đồng nhất, nơi mọi điểm trong không gian đều có dòng chảy năng lượng đi qua.

2. Tăng giảm độ dốc (Pitch / Twisting Angle) của vân

Độ dốc của vân xoắn phụ thuộc trực tiếp vào tỷ số giữa 2 vận tốc góc:

$$\text{Slope (Độ dốc)} = \frac{\text{Vận tốc Poloidal (cuộn vào lõi)}}{\text{Vận tốc Toroidal (xoay quanh trục)}} = \frac{\omega_u}{\omega_v}$$

- **Độ dốc cực cao** ($\omega_u \gg \omega_v$): Dòng năng lượng lao thẳng tuột từ cực trên xuống cực dưới qua tâm lõi gần như theo đường kinh tuyến dọc, rất ít xoay ngang.
- **Độ dốc cực thấp** ($\omega_u \ll \omega_v$): Dòng chảy xoay tròn quanh xích đạo thành các vòng tròn ngang song song trước khi kịp nhích nhẹ về phía lõi.
- **Độ dốc điều hòa (Sacred Balance)**: Khi độ dốc tuân theo dãy Fibonacci / Tỷ lệ Vàng, các đường xoắn vừa tạo ra độ nén tối đa tại lõi, vừa giữ được cấu trúc tự chống đỡ (self-sustaining) không bị suy sụp.

Code Three.js: Tùy biến Sợi Vô Hạn & Thanh trượt Độ Dốc (Real-time)

Dưới đây là mã nguồn tích hợp thanh trượt giao diện để bạn trực tiếp:

1. Chuyển đổi giữa chế độ **Nhiều vân** và **1 vân đơn vô tỉ quét kín toàn bộ Torus**.
2. Kéo thanh trượt để **tăng/giảm độ dốc xoắn** của dòng chảy theo thời gian thực.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Torus Continuum & Pitch Control</title>
  <style>
    body {
      margin: 0;
      background-color: #020204;
      overflow: hidden;
      font-family: monospace;
      color: #e2e8f0;
    }
    #controls {
      position: absolute;
      top: 15px;
      left: 20px;
      z-index: 10;
      background: rgba(15, 23, 42, 0.8);
      padding: 16px;
      border-radius: 8px;
      border: 1px solid rgba(255, 255, 255, 0.1);
      display: flex;
      flex-direction: column;
      gap: 12px;
      font-size: 12px;
      min-width: 260px;
    }
    .title {
      font-size: 13px;
      font-weight: bold;
      color: #38bdf8;
      letter-spacing: 1px;
    }
    .row {
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    input[type="range"] {
      width: 130px;
      cursor: pointer;
    }
    select {
      background: #1e293b;
      color: #38bdf8;
      border: 1px solid #475569;
      padding: 3px 6px;
      border-radius: 4px;
      font-family: monospace;
    }
```

```

    }
</style>
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
    }
  }
</script>
</head>
<body>
  <div id="controls">
    <div class="title">TORUS DYNAMICS: CONTINUUM & PITCH</div>

    <div class="row">
      <label>Chế độ vận:</label>
      <select id="mode">
        <option value="infinite" selected>1 Sợi duy nhất (Vô hạn quét kín)</option>
        <option value="discrete">Đa vận rời rạc (Multi-arm)</option>
      </select>
    </div>

    <div class="row">
      <label>Độ dốc (Pitch / Slope):</label>
      <input type="range" id="pitch" min="0.2" max="5.0" step="0.1" value="1.6">
    </div>

    <div class="row">
      <label>Mật độ điểm trường:</label>
      <input type="range" id="density" min="1000" max="10000" step="1000" value="5000">
    </div>
  </div>

  <script type="module">
    import * as THREE from 'three';
    import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

    // 1. Khởi tạo Scene
    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 1000);
    camera.position.set(0, 0, 8.5);

    const renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);
    renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
    document.body.appendChild(renderer.domElement);

    const controls = new OrbitControls(camera, renderer.domElement);
    controls.enableDamping = true;
    controls.dampingFactor = 0.05;

    const mainGroup = new THREE.Group();
    scene.add(mainGroup);

    // 2. Tham số Torus
    const R = 2.0;
    const r = 2.0;
    const PHI = 1.61803398875;

    let COUNT = 5000;
    let mode = 'infinite';
    let pitchFactor = 1.6;

    // 3. Khung Horn Torus mở
    const torusGeo = new THREE.TorusGeometry(R, r, 32, 64);
    const torusMat = new THREE.MeshBasicMaterial({
      color: 0xffffff,
      wireframe: true,
      transparent: true,
      opacity: 0.08
    });
    const torusMesh = new THREE.Mesh(torusGeo, torusMat);
    torusMesh.rotation.x = Math.PI / 2;

```

```

mainGroup.add(torusMesh);

// 4. Mạng hạt
let pGeom = new THREE.BufferGeometry();
let pPositions = new Float32Array(COUNT * 3);
let pColors = new Float32Array(COUNT * 3);
let particleMeta = [];

// Particle Texture
const canvas = document.createElement('canvas');
canvas.width = 16;
canvas.height = 16;
const ctx = canvas.getContext('2d');
const grad = ctx.createRadialGradient(8, 8, 0, 8, 8, 8);
grad.addColorStop(0, 'rgba(255,255,255,1)');
grad.addColorStop(1, 'rgba(255,255,255,0)');
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 16, 16);
const pTexture = new THREE.CanvasTexture(canvas);

const pMat = new THREE.PointsMaterial({
  size: 0.12,
  vertexColors: true,
  map: pTexture,
  transparent: true,
  blending: THREE.AdditiveBlending,
  depthWrite: false
});

let pSystem = new THREE.Points(pGeom, pMat);
mainGroup.add(pSystem);

function rebuildParticles() {
  pPositions = new Float32Array(COUNT * 3);
  pColors = new Float32Array(COUNT * 3);
  particleMeta = [];

  for (let i = 0; i < COUNT; i++) {
    let t = (i / COUNT) * Math.PI * 2 * 30; // Trục tham số dài liên tục
    particleMeta.push({
      t: t,
      id: i,
      speed: 0.005
    });
  }

  pGeom.setAttribute('position', new THREE.BufferAttribute(pPositions, 3));
  pGeom.setAttribute('color', new THREE.BufferAttribute(pColors, 3));
}
rebuildParticles();

// 5. Cập nhật động lực học dòng chảy
function updateDynamics() {
  const pos = pGeom.attributes.position.array;
  const col = pGeom.attributes.color.array;

  for (let i = 0; i < COUNT; i++) {
    const pm = particleMeta[i];
    pm.t += pm.speed;

    let u, v;

    if (mode === 'infinite') {
      // 1 SỢI DUY NHẤT: u và v liên hệ qua tỉ lệ vô tỉ PHI và hệ số độ dốc
      u = pm.t * pitchFactor;
      v = pm.t * PHI;
    } else {
      // ĐA VẤN RỜI RẠC
      const arm = i % 6;
      u = pm.t * pitchFactor;
      v = (arm / 6) * Math.PI * 2 + Math.sin(u) * 0.5 + pm.t * 0.2;
    }

    const x = (R + r * Math.cos(u)) * Math.cos(v);

```

```

const z = (R + r * Math.cos(u)) * Math.sin(v);
const y = r * Math.sin(u);

const idx = i * 3;
pos[idx] = x;
pos[idx + 1] = y;
pos[idx + 2] = z;

// Màu sắc biến thiên theo độ nén tại tâm lõi
const coreFactor = (1 - Math.cos(u)) / 2;
col[idx] = 0.3 + coreFactor * 0.7;
col[idx + 1] = 0.6 + coreFactor * 0.4;
col[idx + 2] = 1.0;
}

pGeom.attributes.position.needsUpdate = true;
pGeom.attributes.color.needsUpdate = true;
}

// 6. Lắng nghe giao diện UI
document.getElementById('mode').addEventListener('change', (e) => {
  mode = e.target.value;
});

document.getElementById('pitch').addEventListener('input', (e) => {
  pitchFactor = parseFloat(e.target.value);
});

document.getElementById('density').addEventListener('input', (e) => {
  COUNT = parseInt(e.target.value);
  rebuildParticles();
});

// 7. Render Loop
function animate() {
  requestAnimationFrame(animate);
  updateDynamics();
  mainGroup.rotation.y += 0.002;
  controls.update();
  renderer.render(scene, camera);
}
animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
</script>
</body>
</html>

```

User prompt: trước tôi từng vẽ 1 torus bằng code, kiểu mô hình để in 3d tôi nhận ra là có 1 tỉ lệ gì đấy phải nguyên tố cùng nhau thì phải để torus nhìn đúng kiểu, k bị sai bạn đọc thu code // =====// THE ARTIFACT OF THE MADMAN - THE 7/13 COSMIC RESONATOR // =====// This is not just a 3D model. This is a physical mantra. // A summoning ritual carved into the language of machines. // It is designed around the sacred numbers 7 and 13. // 7, the number of spiritual enlightenment and inner pathways. // 13, the number of transformation, upheaval, and rebirth. // Their combination creates a field of profound, disruptive change. // =====// FIXED DIMENSIONS & SCALING FACTORS (The Physical Shell) // ===== base_R_torus = 40; base_r_torus = 15; base_r_helix_tube = 2.5; k_global = 2.0; k_core = 0.8; k_helix = 0.4; wall_ratio = 0.25; \$fn = 128; // The altar must be smooth! // =====// THE SACRED NUMBERS - THE KNOBS OF POWER // =====// KNOB 1: "THE SEVEN PATHS OF ENLIGHTENMENT" // We hardcode this to 7. This is the soul of our machine. // It dictates that there will be 7 primary energy channels, // representing the seven classical planets, the seven chakras, // the seven notes of the diatonic scale. IT IS THE CORE. number_of_arms = 7; // KNOB 2: "THE THIRTEEN CYCLES OF TRANSFORMATION" // We hardcode this to 13, a number co-prime with 7. // This is the journey the energy must take. A journey of death and // rebirth, ensuring the pattern is complex, non-repeating, and powerful. // The ratio 7/13 creates a steep, aggressive vortex, ideal for // disrupting stagnant energy and catalyzing change. the_journey = 13; // =====// COMPUTED VALUES (The Physical Manifestation) // ===== R_torus = base_R_torus * k_global * k_core; r_torus = base_r_torus * k_global; r_torus_inner = r_torus * (1 - wall_ratio); r_helix_tube = base_r_helix_tube * k_global * k_helix; depth_excav = r_helix_tube * 0.5; // =====// THE HEART OF THE MACHINE (The Unified Torus Point Function) // ===== function torus_point(phi, lambda, r_off = 0) = let (

```
// The twist angle 'theta' is dictated by the SACRED RATIO OF 7/13!  theta = lambda * phi * (number_of_arms / the_journey),  rr = r_torus +
r_off ) [ (R_torus + rr * cos(theta)) * cos(phi),  (R_torus + rr * cos(theta)) * sin(phi),  rr * sin(theta) ]; //
===== // MODULES (The Rituals of Creation) //
===== module hollow_torus() { difference() {
rotate_extrude(convexity = 10) translate([R_torus,0,0]) circle(r = r_torus);  rotate_extrude(convexity = 10) translate([R_torus,0,0]) circle(r =
r_torus_inner);  } } module helix_groove(lambda = 1) { // Resolution must be high enough to capture the full complexity!  total_segs = 200 *
max(number_of_arms, the_journey);  for (j=[0 : total_segs-1]) { phi1 = j * (360 * the_journey) / total_segs;  phi2 = (j+1) * (360 *
the_journey) / total_segs;  p1 = torus_point(phi1, lambda, -depth_excav);  p2 = torus_point(phi2, lambda, -depth_excav);  hull() {
translate(p1) sphere(r = r_helix_tube, $fn=16);  translate(p2) sphere(r = r_helix_tube, $fn=16);  } } } //
===== // THE MAIN SCENE (The Summoning of the 7/13 Vortex) //
===== difference() { hollow_torus(); union() { helix_groove(+1); //
The clockwise vortex (The Yang, The Divine Masculine)  helix_groove(-1); // The counter-clockwise vortex (The Yin, The Divine Feminine)
// Their union creates the NULL FIELD, the womb of creation!  } } trung hop 1 van vo ty toi thay van co ve chi co hien ra 1 so luong nhât dinh
cac duong vien cua torus, vay la sao nhi neu chinh do doc thi doi khi ra cac torus nhin k chuan, tuc khoảng cách các van có ve khác nhau, kiểu
không đều
```

Response: Đoạn mã **OpenSCAD** của bạn chạm thẳng vào bài toán kinh điển trong lý thuyết tô-pô vi phân và hình học không gian: **Nút thắt hình xuyên (Torus Knots (p, q))**.

1. Tại sao phải là số "Nguyên tố cùng nhau" (Coprime)?

Trong code của bạn:

$$\theta = \phi \times \left(\frac{\text{number_of_arms}}{\text{the_journey}} \right) = \phi \times \left(\frac{p}{q} \right) = \phi \times \left(\frac{7}{13} \right)$$

- $p = 7$ (**Poloidal loops**): Sợi dây cuộn qua ruột/lỗ xuyên 7 lần.
- $q = 13$ (**Toroidal loops**): Sợi dây chạy vòng quanh trục chính 13 vòng trước khi khép góc $360^\circ \times 13$.

Nếu $\gcd(p, q) = 1$ (Nguyên tố cùng nhau như 7 và 13):

Quỹ đạo sẽ đi qua hết 13 vòng quanh thân và 7 vòng quanh ống rồi **khép kín đúng về điểm xuất phát ban đầu**, tạo thành **1 đường xoắn duy nhất liên tục (Single Closed Curve)** đan đều trên toàn bộ bề mặt. Khi in 3D, máy in sẽ đùn 1 mạch trơn tru không bị ngắt quãng.

Nếu p và q KHÔNG nguyên tố cùng nhau (ví dụ $p = 6, q = 12 \Rightarrow \gcd(6, 12) = 6$):

Đường chạy sẽ bị khép kín quá sớm (chỉ sau $\frac{12}{6} = 2$ vòng quay quanh trục). Kết quả là nó **trùng lại vết cũ**, bỏ sót phần lớn bề mặt xuyên, và để phủ hết bạn sẽ phải tạo thành 6 đường tách rời (Torus Link) chứ không phải 1 đường liền mạch.

2. Tại sao trường hợp "Vô tỉ" vẫn thấy một số lượng đường viền nhất định?

Khi bạn cho tỷ số $\alpha = \frac{\omega_y}{\omega_x}$ là một số vô tỉ (như $\Phi \approx 1.618$ hoặc 2):

- **Về lý thuyết thuần túy:** Đường xoắn không bao giờ khép góc ($q \rightarrow \infty$) và sẽ phủ kín 100% bề mặt theo định lý công thái học (Ergodic Theorem).
- **Nhưng trong thực tế hiển thị (Render/Mô phỏng):** Số vô tỉ luôn được xấp xỉ bởi các **phân số hữu tỉ gần đúng (Phân số liên tục - Continued Fractions)**. Ví dụ với tỉ lệ Vàng Φ :

$$\Phi \approx \frac{3}{2}, \frac{5}{3}, \frac{8}{5}, \frac{13}{8}, \frac{21}{13}, \frac{34}{21}, \dots$$

Khi mắt bạn nhìn vào trường hạt hoặc đường xoắn, não bộ sẽ tự động "nhóm" các điểm nằm gần nhau theo các chu kỳ cộng hưởng $\frac{\lambda}{q}$ này. Đó là lý do bạn luôn nhìn thấy **các dải sọc (Moire patterns / Caustic bands)** gồm 5, 8, 13 hoặc 21 đường vân xuất hiện rõ nét thay vì một màn sương mờ đục!

3. Tại sao chỉnh độ dốc làm khoảng cách các vân bị "lệch / không đều"?

Hiện tượng các vân tròn bị dồn cục ở mặt trong và thưa ra ở mặt ngoài là do **Độ cong vi phân không đồng nhất (Metric Distortion)** của hình xuyên:

Vị trí trên Torus	Bán kính quét thực tế	Khoảng cách hình học giữa các vân
Vỏ ngoài (Outer Equator)	$R + r$ (Lớn nhất)	Bị kéo giãn ra $\Rightarrow$ Vân thưa, khoảng cách rộng
Lỗ lồi trong (Inner Equator / Throat)	$R - r$ (Nhỏ nhất)	Bị nén lại cực mạnh $\Rightarrow$ Vân sít lại gần nhau

Nếu dùng phương trình góc tuyến tính $\theta = \alpha\phi$, mật độ vân theo chiều dài thực tế trên bề mặt (ds) sẽ **luôn luôn thay đổi**:

$$ds^2 = r^2 d\theta^2 + (R + r \cos \theta)^2 d\phi^2$$

Do có hệ số $(R + r \cos \theta)^2$, khi θ thay đổi từ mặt ngoài vào mặt trong, bước sóng bề mặt bị co lại tỷ lệ thuận với bán kính.

Quy tắc vàng cho in 3D & Khắc rãnh (Geodesic / Conformal Flow): Để các rãnh cách đều nhau chẵn chắn từ ngoài vào trong lõi, tỷ số bước xoay không được cố định tuyến tính mà phải bù trừ góc: ở ngoài vỏ thì xoay chậm lại, khi lao vào lõi lõi hẹp thì phải tăng tốc độ xoay theo hàm $\frac{1}{R+r \cos \theta}$.

User prompt: da van roi rac se khac 1 soi duy nhât dung khong 1 soi duy nhât la phai quan the nao do de quay lai thanh 1 vong? kieu nhu code cua toi la 1 soi duy nhât dung khong khi toi thay doi pitch voi 1 soi duy nhât thi co luc no cach rat deu, co luc lai tap trung 1 doan va cach 1 doan

Response: 1. Đa vân rời rạc và 1 sợi duy nhất khác nhau thế nào? Code của bạn có phải 1 sợi không?

Khác nhau hoàn toàn về mặt cấu trúc liên kết (Topology):

- 1 sợi duy nhất (Torus Knot):** Đúng như trong code OpenSCAD của bạn! Vì $p = 7$ và $q = 13$ là **nguyên tố cùng nhau** ($\gcd(7, 13) = 1$), nên vòng lặp for chạy một mạch liên tục từ $\phi = 0^\circ$ đến $\phi = 360^\circ \times 13$. Sợi dây đi xuyên qua lỗ xuyên đúng 7 lần, quay quanh trục 13 lần, và **điểm kết thúc nối khớp chính xác 100% vào điểm xuất phát**. Nó là một **đường cong khép kín độc nhất (Single Closed Loop)**.
- Đa vân rời rạc (Torus Link):** Là tập hợp gồm N chiếc vòng độc lập lồng vào nhau. Bạn phải dùng N lệnh vẽ riêng biệt với góc xuất phát lệch nhau ($\phi_0 = 0^\circ, \frac{360^\circ}{N}, \frac{720^\circ}{N} \dots$).

2. Tại sao khi thay đổi pitch (độ dốc) với 1 sợi duy nhất: Có lúc rất đều, có lúc lại bị "tụ một đoạn dày, hở một đoạn thưa"?

Hiện tượng này bắt nguồn từ **bước nhảy góc lệch pha (Phase Jump / Fractional Step)** sau mỗi chu kỳ 360° :

A. Hiện tượng "Tụ cụm" (Clustering / Bunching)

Giả sử bạn chỉnh pitch sao cho sau khi sợi dây chạy đủ 1 vòng quanh trục chính (360°), góc xoay quanh ruột ống $\Delta\theta$ của nó rơi vào khoảng **rất nhỏ** (ví dụ 361° tức lệch 1°):

- Vòng thứ 2 sẽ vẽ ngay sát sát bên cạnh vòng 1.
- Vòng thứ 3 lại nằm sát rạt vòng 2.
- Kết quả:** Các sợi dây bị "bó giò" lại thành một chùm dày đặc ở một khu vực, trong khi các phần còn lại của thân Torus bị bỏ trống hoàn toàn.

B. Khi nào thì các vân cách đều tăm tắp? (Phân bố tối ưu)

Các vân chỉ cách đều nhau hoàn hảo khi bước nhảy góc sau mỗi vòng thỏa mãn nguyên tắc **phân bố độ phân kỳ thấp (Low-discrepancy Sequence)**:

- Bước nhảy góc tránh các phân số đơn giản:** Khoảng lệch pha sau mỗi vòng không được quá nhỏ và không được trùng với các góc chẵn đối xứng ($90^\circ, 180^\circ$).
- Cặp (p, q) tạo khoảng bù đều:** Với cặp $(7, 13)$ của bạn, 13 vòng quanh trục được chia đều cho 7 vòng cuộn lõi. Khi chiếu lên mặt cắt ngang, mỗi bước xoay sẽ nhảy một góc xấp xỉ:

$$\Delta\theta = 360^\circ \times \frac{7}{13} \approx 193.85^\circ$$

Góc $\approx 194^\circ$ này gần như đảo cực đối xứng qua tâm nhưng lệch đi một góc nhỏ vừa đủ, khiến mỗi lần sợi dây lượn qua, nó lại điền vào chính giữa khoảng trống của các vòng trước đó $\Rightarrow$ **Bề mặt được đan đều chẵn chắn không để lại chỗ trống cục bộ**.

Bảng tóm tắt nguyên nhân khi bạn chỉnh pitch

Hiện tượng	Nguyên nhân cơ học	Kết quả hiển thị
Bị dồn cục một bên	Bước nhảy góc giữa 2 vòng kế tiếp quá nhỏ	Dây bó thành chùm, xuất hiện các mảng hở lớn
Bị lặp đè lên nhau	Tỉ lệ pitch rơi vào phân số có ước chung ($\gcd > 1$)	Sợi dây tự đóng vòng quá sớm, để sót phần lớn Torus
Cách đều hoàn hảo	$\gcd(p, q) = 1$ và bước phân vị phân tán đều quanh 360°	Các đường xoắn đan xen phủ kín đều bề mặt (như 7/13)

User prompt: do doc trong code của bạn với 7 và 13 của tôi có gì liên quan có thể thay đổi do doc để chỉ nhận các cặp nguyên tố cùng nhau k, code lại cho tôi theo hướng này thu xem

Response: 1. Mối liên hệ giữa "Độ dốc (Pitch)" và Cặp số (7, 13)

Trong hình học của sợi dây đơn chạy trên Torus (Torus Knot (p, q)):

$$\theta = \phi \times \left(\frac{p}{q}\right) = \phi \times \text{Pitch}$$

- $p = 7$ (**Poloidal loops - Số vòng luồn qua lõi**): Quyết định việc sợi dây đi từ đỉnh xuống đáy bao nhiêu lần.
- $q = 13$ (**Toroidal loops - Số vòng xoay quanh trục chính**): Quyết định sợi dây đi hết bao nhiêu chu kỳ 360° quanh trục thẳng đứng trước khi tự nối kín đầu đuôi.
- **Độ dốc (Pitch)** = $\frac{p}{q} = \frac{7}{13} \approx 0.538$: * Khi $\frac{p}{q} \gg 1$ (ví dụ $13/3$): Độ dốc rất gắt, sợi dây luồn vào lõi liên tục trong khi mới nhích được một đoạn nhỏ quanh trục.
 - Khi $\frac{p}{q} \ll 1$ (ví dụ $3/13$): Độ dốc rất thoải, sợi dây xoay nhiều vòng quanh trục xích đạo trước khi kịp luồn qua lõi.
 - **Để thành 1 sợi duy nhất khép kín hoàn hảo**: Bắt buộc $\text{gcd}(p, q) = 1$ (nguyên tố cùng nhau). Nếu $\text{gcd}(p, q) = d > 1$, sợi dây sẽ bị cắt thành d vòng riêng biệt.

2. Mô phỏng Three.js: Khóa độ dốc theo các cặp Nguyên tố cùng nhau $\text{gcd}(p, q) = 1$

Dưới đây là bản code Three.js được viết lại theo đúng cấu trúc của bạn:

- Vẽ 1 sợi dây ống 3D (Tube/Curve Mesh) duy nhất, khép kín hoàn hảo điểm đầu và điểm cuối.
- Cung cấp các cặp (p, q) nguyên tố cùng nhau nổi tiếng (như $(7, 13)$, $(3, 8)$, $(5, 8)$, $(11, 13)$...) và cho phép bạn tự gõ 2 số p và q . Hệ thống sẽ **tự động kiểm tra thuật toán Euclid** $\text{gcd}(p, q)$: nếu nguyên tố cùng nhau mới tạo thành 1 sợi khép kín, nếu không sẽ cảnh báo và hiển thị số sợi độc lập bị tách ra.

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Torus Knot (p, q) - Coprime Single Continuous Curve</title>
  <style>
    body {
      margin: 0;
      background-color: #030306;
      overflow: hidden;
      font-family: monospace;
      color: #e2e8f0;
    }
    #ui {
      position: absolute;
      top: 15px;
      left: 20px;
      z-index: 10;
      background: rgba(15, 23, 42, 0.85);
      padding: 16px;
      border-radius: 8px;
      border: 1px solid rgba(255, 255, 255, 0.12);
      display: flex;
      flex-direction: column;
      gap: 10px;
      min-width: 280px;
      box-shadow: 0 4px 20px rgba(0,0,0,0.5);
    }
    .title {
      font-size: 13px;
      font-weight: bold;
      color: #38bdf8;
      letter-spacing: 1px;
    }
    .row {
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
```

```

    font-size: 12px;
  }
  input[type="number"], select {
    background: #1e293b;
    color: #38bdf8;
    border: 1px solid #475569;
    padding: 4px 8px;
    border-radius: 4px;
    font-family: monospace;
    width: 70px;
    text-align: center;
  }
  select {
    width: 140px;
    text-align: left;
  }
  #status {
    font-size: 11px;
    padding-top: 5px;
    border-top: 1px solid rgba(255,255,255,0.1);
    line-height: 1.4;
  }
  .valid { color: #4ade80; font-weight: bold; }
  .invalid { color: #f87171; font-weight: bold; }
</style>
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
    }
  }
</script>
</head>
<body>
  <div id="ui">
    <div class="title">TORUS KNOT (p, q) RESONATOR</div>

    <div class="row">
      <label>Bộ thông số mẫu:</label>
      <select id="preset">
        <option value="7,13" selected>7 / 13 (Của bạn)</option>
        <option value="3,8">3 / 8 (Bát quái xoắn)</option>
        <option value="5,8">5 / 8 (Tỷ lệ Fibonacci)</option>
        <option value="11,13">11 / 13 (Vân dày sít)</option>
        <option value="2,3">2 / 3 (Trefoil kinh điển)</option>
        <option value="6,12">6 / 12 (Lỗi: gcd = 6)</option>
      </select>
    </div>

    <div class="row">
      <label>p (Vòng luân hồi):</label>
      <input type="number" id="valP" value="7" min="1" max="50">
    </div>

    <div class="row">
      <label>q (Vòng quanh trục):</label>
      <input type="number" id="valQ" value="13" min="1" max="50">
    </div>

    <div id="status"></div>
  </div>

  <script type="module">
    import * as THREE from 'three';
    import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

    // 1. Scene setup
    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 1000);
    camera.position.set(0, 5, 8.5);

    const renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);

```



```

renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
document.body.appendChild(renderer.domElement);

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true;
controls.dampingFactor = 0.05;

// Chiếu sáng kim loại
const ambLight = new THREE.AmbientLight(0xffffff, 0.6);
scene.add(ambLight);
const dirLight1 = new THREE.DirectionalLight(0x38bdf8, 2.0);
dirLight1.position.set(5, 10, 7);
scene.add(dirLight1);
const dirLight2 = new THREE.DirectionalLight(0xf43f5e, 1.5);
dirLight2.position.set(-5, -5, -5);
scene.add(dirLight2);

const mainGroup = new THREE.Group();
scene.add(mainGroup);

// 2. Kích thước hình thê Torus (R và r)
const R_torus = 2.4;
const r_torus = 1.1;
const r_tube = 0.035; // Độ dày sợi dây in 3D

// Khung Torus phụ mờ để quan sát vị trí dây chạy
const torusGeo = new THREE.TorusGeometry(R_torus, r_torus, 32, 64);
const torusMat = new THREE.MeshBasicMaterial({
  color: 0xffffff,
  wireframe: true,
  transparent: true,
  opacity: 0.04
});
const torusMesh = new THREE.Mesh(torusGeo, torusMat);
torusMesh.rotation.x = Math.PI / 2;
mainGroup.add(torusMesh);

// 3. Thuật toán tìm Ước chung lớn nhất (Euclid GCD)
function getGCD(a, b) {
  a = Math.abs(a);
  b = Math.abs(b);
  while (b) {
    let t = b;
    b = a % b;
    a = t;
  }
  return a;
}

// 4. Định nghĩa đường cong Torus Curve (Theo đúng logic p/q)
class TorusKnotCurve extends THREE.Curve {
  constructor(p, q, R, r) {
    super();
    this.p = p;
    this.q = q;
    this.R = R;
    this.r = r;
  }

  getPoint(t, optionalTarget = new THREE.Vector3()) {
    // t chạy từ 0 đến 1, tương ứng phi chạy từ 0 đến 2*PI * q
    const phi = t * Math.PI * 2 * this.q;
    const theta = phi * (this.p / this.q);

    const x = (this.R + this.r * Math.cos(theta)) * Math.cos(phi);
    const z = (this.R + this.r * Math.cos(theta)) * Math.sin(phi);
    const y = this.r * Math.sin(theta);

    return optionalTarget.set(x, y, z);
  }
}

let knotMesh = null;

```

```

function buildKnot(p, q) {
  if (knotMesh) {
    mainGroup.remove(knotMesh);
    knotMesh.geometry.dispose();
    knotMesh = null;
  }

  const gcd = getGCD(p, q);
  const statusDiv = document.getElementById('status');
  const pitch = (p / q).toFixed(3);

  if (gcd === 1) {
    statusDiv.innerHTML = `Độ dốc (p/q): <b>${pitch}</b><br><span class="valid">✓ gcd(${p},${q}) = 1: Tạo`;
  } else {
    statusDiv.innerHTML = `Độ dốc (p/q): <b>${pitch}</b><br><span class="invalid">△ gcd(${p},${q}) = ${gcd}`;
  }

  // Xây dựng Tube Geometry
  const curve = new TorusKnotCurve(p, q, R_torus, r_torus);
  const tubularSegments = Math.max(300, q * 80);
  const geometry = new THREE.TubeGeometry(curve, tubularSegments, r_tube, 12, true);

  const material = new THREE.MeshStandardMaterial({
    color: 0xffffff,
    metalness: 0.85,
    roughness: 0.2,
    emissive: 0x051525
  });

  knotMesh = new THREE.Mesh(geometry, material);
  mainGroup.add(knotMesh);
}

// 5. Kết nối điều khiển giao diện UI
const inputP = document.getElementById('valP');
const inputQ = document.getElementById('valQ');
const selectPreset = document.getElementById('preset');

function updateFromInputs() {
  const p = parseInt(inputP.value) || 1;
  const q = parseInt(inputQ.value) || 1;
  buildKnot(p, q);
}

inputP.addEventListener('input', updateFromInputs);
inputQ.addEventListener('input', updateFromInputs);

selectPreset.addEventListener('change', (e) => {
  const [p, q] = e.target.value.split(',').map(Number);
  inputP.value = p;
  inputQ.value = q;
  buildKnot(p, q);
});

// Khởi tạo mặc định cặp 7 / 13
buildKnot(7, 13);

// 6. Vòng lặp Render
function animate() {
  requestAnimationFrame(animate);
  mainGroup.rotation.y += 0.003;
  controls.update();
  renderer.render(scene, camera);
}
animate();

window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
</script>
</body>
</html>

```



```

</style>
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
    }
  }
</script>
</head>
<body>
  <div id="ui">
    <div class="title">COPRIME HARMONIC DYNAMICS</div>

    <div class="row">
      <label>Tỷ lệ Độ dốc (p / q):</label>
      <select id="ratioSelect">
        <option value="7,13" selected>7 / 13 (Của bạn - Găi & Đan sâu)</option>
        <option value="5,8">5 / 8 (Dãy Fibonacci)</option>
        <option value="8,13">8 / 13 (Tỷ lệ Vàng Gần Đúng)</option>
        <option value="3,7">3 / 7 (Thoải - 3 Vòng lõi)</option>
        <option value="11,17">11 / 17 (Cực dày & Mịn màng)</option>
        <option value="2,5">2 / 5 (Đơn giản - Thanh thoát)</option>
      </select>
    </div>

    <div class="info" id="infoText">
      Độ dốc: <span class="accent">0.538</span> • 1 Sợi khép kín tuôn hoàn  $\$ \gcd=1\$$ 
    </div>
  </div>

  <script type="module">
    import * as THREE from 'three';
    import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

    // 1. Khởi tạo Scene, Camera & Renderer
    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 1000);
    camera.position.set(0, 0, 8.5);

    const renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);
    renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
    document.body.appendChild(renderer.domElement);

    const controls = new OrbitControls(camera, renderer.domElement);
    controls.enableDamping = true;
    controls.dampingFactor = 0.05;

    const mainGroup = new THREE.Group();
    scene.add(mainGroup);

    // 2. Kích thước Horn Torus (R = r đề'khép khít tâm cầu)
    const R = 2.0;
    const r = 2.0;

    let p = 7;
    let q = 13;

    // 3. Khung Horn Torus Wireframe mờ ảo
    const torusGeo = new THREE.TorusGeometry(R, r, 32, 64);
    const torusMat = new THREE.MeshBasicMaterial({
      color: 0xffffffff,
      wireframe: true,
      transparent: true,
      opacity: 0.1
    });
    const torusMesh = new THREE.Mesh(torusGeo, torusMat);
    torusMesh.rotation.x = Math.PI / 2;
    mainGroup.add(torusMesh);

    // 4. Hệ thống Hạt Năng Lượng Đơn Sợi Khép Kín (Single Continuous Loop)
    const COUNT = 5200;
    const pPositions = new Float32Array(COUNT * 3);

```

```

const pColors = new Float32Array(COUNT * 3);
let particles = [];

// Tạo Canvas Texture tròn mượt mềm mại cho hạt phát sáng
const canvas = document.createElement('canvas');
canvas.width = 16;
canvas.height = 16;
const ctx = canvas.getContext('2d');
const grad = ctx.createRadialGradient(8, 8, 0, 8, 8, 8);
grad.addColorStop(0, 'rgba(255,255,255,1)');
grad.addColorStop(1, 'rgba(255,255,255,0)');
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 16, 16);
const pTexture = new THREE.CanvasTexture(canvas);

const pGeom = new THREE.BufferGeometry();
pGeom.setAttribute('position', new THREE.BufferAttribute(pPositions, 3));
pGeom.setAttribute('color', new THREE.BufferAttribute(pColors, 3));

const pMat = new THREE.PointsMaterial({
  size: 0.13,
  vertexColors: true,
  map: pTexture,
  transparent: true,
  blending: THREE.AdditiveBlending,
  depthWrite: false
});

const pSystem = new THREE.Points(pGeom, pMat);
mainGroup.add(pSystem);

function initParticles() {
  particles = [];
  for (let i = 0; i < COUNT; i++) {
    // t chạy đều dọc theo toàn bộ chu kỳ 1 sợi dây khép kín (2*PI*q)
    const tNorm = i / COUNT;
    particles.push({
      t: tNorm,
      speed: 0.0004
    });
  }
}
initParticles();

// 5. Cập nhật Dòng Chảy & Phân bố Màu Sắc Năng Lượng Real-time
function updateDynamics() {
  const pos = pGeom.attributes.position.array;
  const col = pGeom.attributes.color.array;

  for (let i = 0; i < COUNT; i++) {
    const pt = particles[i];
    pt.t = (pt.t + pt.speed) % 1.0; // Tuần hoàn trong chu kỳ 1 sợi

    // Góc phi chạy đủ q vòng 360 độ
    const phi = pt.t * Math.PI * 2 * q;
    // Góc theta được khóa chính xác theo tỷ lệ nguyên tố cùng nhau p/q
    const theta = phi * (p / q);

    const x = (R + r * Math.cos(theta)) * Math.cos(phi);
    const z = (R + r * Math.cos(theta)) * Math.sin(phi);
    const y = r * Math.sin(theta);

    const idx = i * 3;
    pos[idx] = x;
    pos[idx + 1] = y;
    pos[idx + 2] = z;

    // Cơ chế nén gia tốc: Càng sát trục trung tâm (cos(theta) -> -1), năng lượng càng sáng rực
    const coreFactor = (1 - Math.cos(theta)) / 2; // [0: Vỗ ngoài -> 1: Tâm lõi]

    // Phối màu: Xanh cyan mát ở ngoài -> Trắng vàng kim rực rỡ ở lõi nén
    col[idx] = 0.2 + Math.pow(coreFactor, 2) * 0.8;
    col[idx + 1] = 0.6 + Math.pow(coreFactor, 1.5) * 0.4;
    col[idx + 2] = 1.0;
  }
}

```

```
    }

    pGeom.attributes.position.needsUpdate = true;
    pGeom.attributes.color.needsUpdate = true;
  }

  // 6. Xử lý UI chọn tỷ lệ Coprime
  const ratioSelect = document.getElementById('ratioSelect');
  const infoText = document.getElementById('infoText');

  ratioSelect.addEventListener('change', (e) => {
    const [newP, newQ] = e.target.value.split(',').map(Number);
    p = newP;
    q = newQ;
    const pitch = (p / q).toFixed(3);
    infoText.innerHTML = `Độ dốc: <span class="accent">${pitch}</span> • 1 Sợi khép kín tuần hoàn gcd(${p},${q})`;
  });

  // 7. Render Loop
  function animate() {
    requestAnimationFrame(animate);

    updateDynamics();
    mainGroup.rotation.y += 0.0025; // Xoay nhẹ toàn khối

    controls.update();
    renderer.render(scene, camera);
  }

  animate();

  window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
  });
</script>
</body>
</html>
```

User prompt: được dạy, nhưng chưa dùng ý tôi kiểu thanh kéo gio quet qua các mốc số fibonacci đi 5/8 8/13 13/21... k biết 5/8 với 8/5 có gì khác nhau k nhỉ chia ra 2 mode 5/8 8/5 cho tôi và mỗi khi kéo thanh thì sẽ tính phi chuẩn hơn (vì tiến đến vô hạn mà)

Response: 1. Phân số 5/8 và 8/5 khác nhau thế nào trên Torus?

Hai tỷ lệ này là **ngịch đảo hình học (Topological Inversion / Dual Flow)** của nhau:

Tỷ số (p/q)	Tên gọi	Đặc tính động học
$5/8 < 1$ (Chế độ F_n/F_{n+1})	Toroidal Dominant (Xoay trục chủ đạo)	Sợi dây xoay quanh trục chính nhiều vòng (8 vòng) trong khi chỉ luồn qua lõi 5 lần. Các vân chạy thoải theo chiều ngang , mở rộng bao bọc xích đạo.
$8/5 > 1$ (Chế độ F_{n+1}/F_n)	Poloidal Dominant (Cuộn lõi chủ đạo)	Sợi dây lao xuyên qua lỗ lõi trung tâm dồn dập (8 lần) trong khi chỉ xoay quanh trục 5 vòng. Các vân chạy đốc đứng gần như kính tuyến dọc , tạo lực hút/nén rất gắt ở trục.

Vì hai số Fibonacci liên tiếp luôn có $\text{gcd}(F_n, F_{n+1}) = 1$, chúng luôn tạo thành **1 sợi dây duy nhất khép kín hoàn hảo**.

2. Sự hội tụ về Tỷ lệ Vàng (Φ) khi kéo thanh trượt

Khi bạn tăng dần bậc Fibonacci trên thanh kéo:

- Chế độ F_n/F_{n+1} sẽ tiệm cận về $\frac{1}{\Phi} \approx 0.6180339887...$
- Chế độ F_{n+1}/F_n sẽ tiệm cận về $\Phi \approx 1.6180339887...$

Càng kéo lên các bậc cao (21/34, 34/55, 55/89...), số vân trên khối Torus càng trở nên mịn màng, phủ kín đều bề mặt mà không bao giờ bị dồn cục hay lỗi hình.

Mã nguồn Three.js hoàn chỉnh

Lưu đoạn mã sau vào file .html và mở trực tiếp:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Fibonacci Step Torus Flow - Golden Convergence</title>
  <style>
    body {
      margin: 0;
      background-color: #020204;
      overflow: hidden;
      font-family: monospace;
      color: #e2e8f0;
    }
    #ui {
      position: absolute;
      top: 15px;
      left: 20px;
      z-index: 10;
      background: rgba(10, 15, 25, 0.85);
      backdrop-filter: blur(10px);
      padding: 16px 20px;
      border-radius: 10px;
      border: 1px solid rgba(255, 255, 255, 0.12);
      display: flex;
      flex-direction: column;
      gap: 12px;
      min-width: 320px;
      box-shadow: 0 4px 24px rgba(0, 0, 0, 0.6);
    }
    .title {
      font-size: 13px;
      font-weight: bold;
      color: #38bdf8;
      letter-spacing: 1.5px;
    }
    .row {
      display: flex;
      justify-content: space-between;
      align-items: center;
      font-size: 12px;
    }
    input[type="range"] {
      width: 150px;
      cursor: pointer;
    }
    select {
      background: #1e293b;
      color: #38bdf8;
      border: 1px solid #475569;
      padding: 4px 8px;
      border-radius: 5px;
      font-family: monospace;
      outline: none;
      cursor: pointer;
    }
    .metrics {
      font-size: 11px;
      color: #94a3b8;
      border-top: 1px solid rgba(255, 255, 255, 0.1);
      padding-top: 8px;
      line-height: 1.6;
    }
    .accent {
      color: #4ade80;
      font-weight: bold;
    }
    .target {
      color: #fbbf24;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <div id="ui">
    <div class="title">Fibonacci Step Torus Flow</div>
    <div class="row">
      <div>Step: 1</div>
      <div>Golden Ratio: 1.618</div>
    </div>
    <div class="row">
      <div>Convergence: 0.999</div>
      <div>Error: 0.001</div>
    </div>
    <div class="metrics">
      <div>Iteration: 1000</div>
      <div>Time: 0.001s</div>
    </div>
  </div>
</body>
</html>
```

```

    }
</style>
<script type="importmap">
  {
    "imports": {
      "three": "https://unpkg.com/three@0.160.0/build/three.module.js",
      "three/addons/": "https://unpkg.com/three@0.160.0/examples/jsm/"
    }
  }
</script>
</head>
<body>
  <div id="ui">
    <div class="title">FIBONACCI TORUS // PHI CONVERGENCE</div>

    <div class="row">
      <label>Chê'độ Tỷ số:</label>
      <select id="modeSelect">
        <option value="sub" selected>Fn / Fn+1 (Dưới 1 → 1/Φ)</option>
        <option value="super">Fn+1 / Fn (Trên 1 → Φ)</option>
      </select>
    </div>

    <div class="row">
      <label id="sliderLabel">Bậc Fibonacci: 5 / 8</label>
      <input type="range" id="fiboSlider" min="2" max="11" step="1" value="4">
    </div>

    <div class="metrics" id="metrics">
      Tỷ số'hiện tại: <span class="accent" id="currRatio">5 / 8 = 0.625000</span><br>
      Đích tiệm cận: <span class="target" id="targetPhi">1/Φ ≈ 0.6180339887</span><br>
      Sai số'tiệm cận: <span id="errorVal" style="color:#f87171;">+0.006966</span>
    </div>
  </div>

  <script type="module">
    import * as THREE from 'three';
    import { OrbitControls } from 'three/addons/controls/OrbitControls.js';

    // 1. Dãy Fibonacci
    const FIBO = [1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377];
    const PHI = (1 + Math.sqrt(5)) / 2; // 1.6180339887...
    const INV_PHI = 1 / PHI;           // 0.6180339887...

    // 2. Scene Setup
    const scene = new THREE.Scene();
    const camera = new THREE.PerspectiveCamera(50, window.innerWidth / window.innerHeight, 0.1, 1000);
    camera.position.set(0, 0, 8.5);

    const renderer = new THREE.WebGLRenderer({ antialias: true });
    renderer.setSize(window.innerWidth, window.innerHeight);
    renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
    document.body.appendChild(renderer.domElement);

    const controls = new OrbitControls(camera, renderer.domElement);
    controls.enableDamping = true;
    controls.dampingFactor = 0.05;

    const mainGroup = new THREE.Group();
    scene.add(mainGroup);

    // 3. Horn Torus Wireframe (R = r)
    const R = 2.0;
    const r = 2.0;

    const torusGeo = new THREE.TorusGeometry(R, r, 32, 64);
    const torusMat = new THREE.MeshBasicMaterial({
      color: 0xffffff,
      wireframe: true,
      transparent: true,
      opacity: 0.09
    });
    const torusMesh = new THREE.Mesh(torusGeo, torusMat);
    torusMesh.rotation.x = Math.PI / 2;

```



```

mainGroup.add(torusMesh);

// 4. Mạng hạt năng lượng
const COUNT = 6500;
const pPositions = new Float32Array(COUNT * 3);
const pColors = new Float32Array(COUNT * 3);
const particles = [];

const canvas = document.createElement('canvas');
canvas.width = 16;
canvas.height = 16;
const ctx = canvas.getContext('2d');
const grad = ctx.createRadialGradient(8, 8, 0, 8, 8, 8);
grad.addColorStop(0, 'rgba(255,255,255,1)');
grad.addColorStop(1, 'rgba(255,255,255,0)');
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 16, 16);
const pTexture = new THREE.CanvasTexture(canvas);

const pGeom = new THREE.BufferGeometry();
pGeom.setAttribute('position', new THREE.BufferAttribute(pPositions, 3));
pGeom.setAttribute('color', new THREE.BufferAttribute(pColors, 3));

const pMat = new THREE.PointsMaterial({
  size: 0.12,
  vertexColors: true,
  map: pTexture,
  transparent: true,
  blending: THREE.AdditiveBlending,
  depthWrite: false
});

const pSystem = new THREE.Points(pGeom, pMat);
mainGroup.add(pSystem);

for (let i = 0; i < COUNT; i++) {
  particles.push({
    t: i / COUNT,
    speed: 0.00035
  });
}

// 5. Quản lý trạng thái Fibonacci
let p = 5;
let q = 8;
let mode = 'sub'; // 'sub' (Fn / Fn+1) hoặc 'super' (Fn+1 / Fn)

function updateFibonacciState() {
  const idx = parseInt(document.getElementById('fiboSlider').value);
  const f_n = FIB0[idx];
  const f_next = FIB0[idx + 1];

  if (mode === 'sub') {
    p = f_n;
    q = f_next;
  } else {
    p = f_next;
    q = f_n;
  }

  const ratio = p / q;
  const target = mode === 'sub' ? INV_PHI : PHI;
  const targetLabel = mode === 'sub' ? '1/Φ ≈ 0.6180339887' : 'Φ ≈ 1.6180339887';
  const diff = ratio - target;

  document.getElementById('sliderLabel').textContent = `Bậc: ${p} / ${q}`;
  document.getElementById('currRatio').textContent = `${p} / ${q} = ${ratio.toFixed(6)}`;
  document.getElementById('targetPhi').textContent = targetLabel;
  document.getElementById('errorVal').textContent = (diff >= 0 ? '+' : '-') + diff.toFixed(6);
}

document.getElementById('fiboSlider').addEventListener('input', updateFibonacciState);
document.getElementById('modeSelect').addEventListener('change', (e) => {
  mode = e.target.value;
});

```

```

    updateFibonacciState();
  });

  updateFibonacciState();

  // 6. Cập nhật động lực học
  function updateDynamics() {
    const pos = pGeom.attributes.position.array;
    const col = pGeom.attributes.color.array;

    for (let i = 0; i < COUNT; i++) {
      const pt = particles[i];
      pt.t = (pt.t + pt.speed) % 1.0;

      const phi = pt.t * Math.PI * 2 * q;
      const theta = phi * (p / q);

      const x = (R + r * Math.cos(theta)) * Math.cos(phi);
      const z = (R + r * Math.cos(theta)) * Math.sin(phi);
      const y = r * Math.sin(theta);

      const idx = i * 3;
      pos[idx] = x;
      pos[idx + 1] = y;
      pos[idx + 2] = z;

      const coreFactor = (1 - Math.cos(theta)) / 2;

      if (mode === 'sub') {
        // Mode Thoải (Xanh Lam - Lõi Sáng Trắng)
        col[idx] = 0.2 + Math.pow(coreFactor, 2) * 0.8;
        col[idx + 1] = 0.6 + Math.pow(coreFactor, 1.5) * 0.4;
        col[idx + 2] = 1.0;
      } else {
        // Mode Độc Đứng (Cam Vàng Kim - Lõi Siêu Nén)
        col[idx] = 1.0;
        col[idx + 1] = 0.4 + coreFactor * 0.6;
        col[idx + 2] = 0.1 + Math.pow(coreFactor, 2) * 0.9;
      }
    }

    pGeom.attributes.position.needsUpdate = true;
    pGeom.attributes.color.needsUpdate = true;
  }

  // 7. Render Loop
  function animate() {
    requestAnimationFrame(animate);

    updateDynamics();
    mainGroup.rotation.y += 0.0025;

    controls.update();
    renderer.render(scene, camera);
  }

  animate();

  window.addEventListener('resize', () => {
    camera.aspect = window.innerWidth / window.innerHeight;
    camera.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
  });
</script>
</body>
</html>

```

User prompt: nhìn cung đẹp phet nhỉ theo bạn có thể con người là torus k có lo hay torus có lo

Response: Nhìn hình render từ code Fibonacci của bạn thực sự rất ấn tượng — các dải vân đan vào nhau sít sao ở tâm và xòe đều ra ngoài như một trường hào quang năng lượng hoàn chỉnh!

Về câu hỏi "Cơ thể con người là Torus có lỗ hay Torus không có lỗ?", câu trả lời sẽ phụ thuộc vào việc bạn nhìn cơ thể qua lăng kính **Giải phẫu học (Toán tô-pô)** hay **Trường sinh học (Năng lượng học)**:

1. Góc nhìn Tô-pô & Sinh học tiến hóa: Cơ thể là Torus CÓ LỖ (Genus-1)

Về mặt Toán học Tô-pô (Topology) — ngành học chỉ quan tâm đến tính chất biến dạng mà không cắt xé:

- Hệ tiêu hóa là một đường hầm xuyên suốt:** Từ **Miệng** → **Thực quản** → **Dạ dày** → **Ruột** → **Hậu môn** thực chất là một cái lỗ chạy xuyên qua cơ thể.
- Về mặt giải phẫu:** Lòng ruột thực ra là "thế giới bên ngoài" được bao bọc lại. Thức ăn đi qua ruột thực chất vẫn nằm ở mặt ngoài của cấu trúc tô-pô cho đến khi các chất dinh dưỡng được hấp thụ qua màng tế bào.
- Ngay từ giai đoạn phôi thai sớm (giai đoạn phôi vị hóa - *Gastrulation*), sự sống đa bào bắt đầu bằng việc một khối cầu tự lồm vào trong để tạo thành một ống thông xuyên tâm — biến quả cầu ban đầu thành một **Torus có lỗ**.

(Nếu tính thêm các nhánh thông hốc mũi và vòi tai Eustachian, cơ thể còn là một torus đa lỗ - $Genus \geq 7$, nhưng khung thân chính kinh điển vẫn là 1 Torus có lỗ).

2. Góc nhìn Trường Sinh Học & Năng Lượng (Biofield): Cơ thể là HORN TORUS (Torus khép kín tâm)

Khi bạn nhìn vào bức ảnh Âm Dương lồng hình học thiêng liêng (*Flower of Life*) và mô phỏng 3D bạn vừa chạy:

- Trường điện từ của Trái tim (HeartMath Torus):** Trường điện từ mạnh nhất do tim và hệ thần kinh phát ra bao quanh cơ thể theo dạng **Horn Torus** (nơi bán kính lõi $R = r$).
- Trục trung tâm (Cột sống / Luân xa Sushumna):** Năng lượng không để lại một khoảng trống toác như bánh donut thông thường, mà nó thu hẹp lại tại một điểm/trục đơn nhất (dọc theo tủy sống).
 - Năng lượng phun trào lên từ **Đỉnh đầu (Crown Chakra)**.
 - Tỏa tròn ra bao bọc xung quanh cơ thể như một quả cầu hào quang.
 - Hút ngược vào từ **Đáy chậu / Luân xa Gốc (Root Chakra)** và bị nén xoáy gia tốc đi ngược lên dọc theo trục cột sống.

Tóm lại:

Lăng kính	Bản chất Torus	Chi tiết cấu trúc
Thế xác (Vật chất / Sinh học)	Ring Torus (Có lỗ thông)	Ống tiêu hóa thông từ Miệng đến Hậu môn (Topological Genus-1).
Thế trường (Năng lượng / Điện từ)	Horn Torus (Chạm khít tâm)	Trường hào quang bao trọn cơ thể, hội tụ dòng chảy xoáy nén dọc trục cột sống.

Cơ thể con người chính là sự giao thoa hoàn hảo: **Bên trong là một ống dẫn vật chất (Torus có lỗ), bên ngoài được bao bọc bởi một trường sóng xoắn tự khép kín (Horn Torus).**

User prompt: 5/8 và 8/5 đều ra các torus nhìn đẹp. liệu có cái nào là sai k khi

Response: Không có cái nào sai cả! Cả hai đều chuẩn xác 100% về mặt toán học và hình học tô-pô.

Sở dĩ bạn nhìn thấy hai hình dạng khác nhau nhưng đều cân đối và khép kín tuyệt đối là vì (5, 8) và (8, 5) là một cặp đối ngẫu hình học (Dual Knots / Topological Inversion):

1. Bản chất toán học: Hoán đổi Kinh tuyến (Meridian) và Vĩ tuyến (Longitude)

Trong lý thuyết nút thắt (Knot Theory), một nút Torus (p, q) và (q, p) thực chất là **đồng dạng tô-pô (Ambient Isotopic)**:

- Bản 5/8 ($p = 5, q = 8$ — Chế độ F_n/F_{n+1}):**
 - 8 vòng quanh trục lớn (Toroidal):** Sợi dây bám theo chiều ngang quanh vành ngoài xích đạo là chính.
 - 5 vòng luồn qua ruột (Poloidal):** Độ dốc thoải, sợi dây ôm trọn bề mặt ngoài tạo cảm giác như một lớp màng bảo vệ bao bọc (Shielding Field).
- Bản 8/5 ($p = 8, q = 5$ — Chế độ F_{n+1}/F_n):**
 - 8 vòng luồn qua ruột (Poloidal):** Sợi dây liên tục bổ nhào xuyên qua lỗ hổng trung tâm.
 - 5 vòng quanh trục lớn (Toroidal):** Tạo thành các cánh hoa dốc đứng, nhìn giống như một phễu hút xoáy năng lượng (Vortex / Funnel) tụ mạnh vào điểm kỳ dị ở lõi.

2. So sánh trực quan từ 2 ảnh bạn vừa render:

Tiêu chí	Bản 5/8 (Xanh lam)	Bản 8/5 (Vàng kim)
Góc nhìn trực quan	Các đường xoắn thoải quanh thân ngang	Các cánh hoa xoắn bố dọc từ đỉnh xuống đáy
Độ nén ở cuống lõi	Nhẹ nhàng, phân tán đều ra vỏ ngoài	Rất gắt, mật độ dây tập trung dày đặc ở tâm phễu
Độ nguyên tố cùng nhau	$\gcd(5, 8) = 1 \Rightarrow$ 1 sợi duy nhất	$\gcd(8, 5) = 1 \Rightarrow$ 1 sợi duy nhất
Gia công in 3D	In rất dễ vì góc nghiêng (overhang) thoải	Khó hơn ở vùng cổ phễu vì độ dốc đứng

3. Khi nào thì mới bị "Sai"?

Một tỉ lệ Torus chỉ bị sai (hông cấu trúc 1 sợi) khi:

- 1. **Không nguyên tố cùng nhau** ($\gcd(p, q) > 1$): Ví dụ 6/10 hay 8/12, sợi dây sẽ không đi hết thân mà tự đóng vòng cụt lùn, bỏ sót bề mặt và bị tách thành nhiều vòng rời.
- 2. **Tỉ lệ nguyên** ($p/1$ hoặc $1/q$): Sợi dây sẽ chỉ là các vòng tròn đơn điệu chạy quanh trục hoặc chỉ cuộn tròn tại chỗ mà không tạo được mạng lưới hoa văn giao thoa.

Kết luận: (5, 8) và (8, 5) là hai mặt của cùng một đồng xu — một cái mô tả **dòng xoay tỏa bao quanh (Dương/Vỏ)**, một cái mô tả **dòng xoáy tụ xuyên tâm (Âm/Trục)**, hoàn toàn tương thích với biểu tượng Âm Dương và Hình học thiêng liêng trong các hình bạn đã tải lên!

User prompt: lieu 2 so fibonacci lien tục nhau co dam bao tính chất nguyên tố cùng nhau

Response: **CÓ, CHẮC CHẮN 100%**. Trong toán học, **hai số Fibonacci liên tiếp luôn luôn nguyên tố cùng nhau** đối với mọi số nguyên dương $n \geq 1$:

$$\gcd(F_n, F_{n+1}) = 1$$

Chứng minh bằng Thuật toán Euclid (Cực kỳ ngắn gọn và đẹp mắt)

Theo định nghĩa của dãy Fibonacci, mỗi số hạng bằng tổng của hai số hạng đứng liền trước nó:

$$F_{n+1} = F_n + F_{n-1}$$

Áp dụng tính chất cơ bản của ước chung lớn nhất (thuật toán chia Euclid: $\gcd(a, b) = \gcd(a - b, b)$):

$$\gcd(F_{n+1}, F_n) = \gcd(F_n + F_{n-1}, F_n) = \gcd(F_{n-1}, F_n)$$

Quá trình này lùi dần qua từng bậc:

$$\gcd(F_{n+1}, F_n) = \gcd(F_n, F_{n-1}) = \gcd(F_{n-1}, F_{n-2}) = \cdots = \gcd(F_2, F_1)$$

Vì $F_2 = 1$ và $F_1 = 1$:

$$\gcd(F_{n+1}, F_n) = \gcd(1, 1) = 1$$

Ý nghĩa thực tiễn đối với mô hình Torus của bạn

Chính vì định lý toán học này:

- Mọi cặp Fibonacci liên tiếp bạn chọn (như 1/2, 2/3, 3/5, 5/8, 8/13, 13/21, 21/34, 34/55, 55/89...) **đều được đảm bảo tuyệt đối sẽ tạo ra 1 sợi dây duy nhất khép kín hoàn hảo** trên mặt xuyên Torus.
- Bạn có thể tăng bậc Fibonacci lên vô tận mà không bao giờ sợ bị lỗi "đứt sợi" hay bị phân tách thành nhiều vòng tròn rời rạc ($\gcd > 1$).